

Design and Performance Evaluation of an AES-128 Hardware Accelerator in a MicroBlaze V FPGA System

Venkatesh Maddukuri
School of Electronic Engineering
Dublin City University
 Dublin 9, D09V209, Ireland
 venkatesh.maddukuri2@mail.dcu.ie

Abstract

In this paper, the AES-128 encryption presents the design, implementation, and performance evaluation of an AES-128 hardware accelerator integrated with a MicroBlaze V processor-based FPGA system core is implemented in dedicated hardware and connected to the processor via an AXI4-Lite peripheral, while a software AES-128 implementation running on the MicroBlaze V softcore processor is used as a performance baseline. Both software and hardware implementations are evaluated using identical 128bit plaintext and key inputs, with ciphertext comparison used to verify functional correctness of both software and hardware implementations. An AXI timer setup at 100 MHz timer frequency to measure clock cycles, the hardware implementation required 1,196 cycles with a measured latency of 11.960 μ s and throughput of 10.702 Mbps, whereas the software implementation required 39,888 cycles, 398.880 μ s latency, and achieved 0.320 Mbps throughput. The resulting hardware acceleration highlights a measured speedup of approximately 33.35x, highlighting the performance gain from moving AES-128 encryption to a dedicated FPGA hardware.

Keywords—

AES-128, AXI4-Lite, FPGA, hardware acceleration, MicroBlaze V, performance evaluation.

I. INTRODUCTION

The Advanced Encryption Standard(AES) is mostly used algorithm to keep the information safe. It is used in lots of systems. The Advanced Encryption Standard works with blocks of data with 128 bits. It can use key length of 128 bits, 192 bits or 256 bits[1]. The Advanced Encryption Standard can be implemented in purely software, encryption executed by a processor requires repeated arithmetic operations and data-transformation operations, This can slow down systems that do not have a lot of power. There is another way to do The Advanced Encryption Standard, It is called Field-programmable gate arrays (FPGAs). This is a kind of hardware that can do the hard work which computationally intensive cryptographic operations of the Advanced Encryption Standard. This means the processor can still be in control and the system can still be flexible.

In processor based embedded systems, the execution time of cryptographic algorithms takes long, this affect overall system performance, particularly when encryption should be performed repeatedly or under quick timing constraints. Hardware acceleration address this limitation by designing and implementing digitally intensive operations as dedicated digital logic rather than executing each operation sequentially as present in software instructions[5], [7], [8]. For AES-128, operations associated with the encryption rounds can therefore be implemented directly within FPGA logic, while the processor should remain responsible for system control, data transfer, and application-level tasks. Previous hardware/software co-design and FPGA-based AES implementations have demonstrated the potential of this approach for improving cryptographic processing performance [3], [4]. A hardware/software architecture of this type provides a useful platform for evaluating the performance improvement obtained by accelerating cryptographic processing while maintaining software-based control of the overall system.

This research investigates hardware acceleration of the AES-128 encryption scheme on an FPGA architecture taking the MicroBlaze V processor [2]. The AES-128 encryption engine was designed in Verilog and interfaced with the processor via a customized AXI4-Lite peripheral design that enabled the transfer of the 128 bit input message and the encryption key from the MicroBlaze V processor to the accelerator and the recovery of the 128 bit encrypted message. Additionally, the AES-128 algorithm was implemented in software and run on the same processor to obtain a benchmark of its performance against the hardware implementation. An AXI Timer was Placed to measure the number of execution cycles takes to complete for both implementations.

The experimental evaluation showed that the output ciphertext was same in both hardware and software implementations when the same test inputs were supplied to the two systems. In the mentioned comparative analysis, the number of execution cycles needed for the hardware implementation was 1,196 cycles while the software implementation consumed 39,888 cycles. At the 100 MHz timer frequency, such cycle counts translate into the latencies of 11.960 μ s and 398.880 μ s and the throughputs of 10.702 Mbps and 0.320 Mbps for the hardware and software implementations, correspondingly. This speed of hardware

resulting 128bit ciphertext to the processor through the AXI-connected peripheral.

C. AXI4-Lite Interface and Register Map

The AES-128 hardware accelerator communicates with the MicroBlaze V processor through a custom designed AXI4-Lite slave interface. Since the implemented AXI data interface uses 32-bit words for data transfer, both the plaintext and encryption key of size 128 bits each are segmented into four 32-bit words and loaded into respective registers in the peripheral. Similarly, the final 128bit ciphertext output is loaded in four 32-bit output registers that are accessible by the processor. Control and status registers are utilized to trigger the encryption process and know the point at which the hardware accelerator finishes its processing task.

TABLE. I. AES-128 Accelerator Register Map

Register Group	Width	Function
Plaintext Registers	128 bits	Stores plaintext input
Key Registers	128 bits	Stores AES-128 encryption key
Ciphertext Registers	128 bits	Stores encrypted output
Control Register	32 bits	Starts encryption
Status Register	32 bits	Indicates encryption completion

The AES-128 AXI peripheral is memory-mapped into the MicroBlaze V address space with a base address of 0x80000000.

D. MicroBlaze V Software Implementation

Software implementation of AES-128 has been performed in C language and run on the MicroBlaze V processor. Software implementation is able to perform AES-128 encryption operation utilizing the processor instruction set without the use of hardware AES accelerator. Software implementation takes the same 128bit plaintext input as well as the same 128bit encryption key used by the hardware implementation and outputs 128bit ciphertext. Using the same input data for both software and hardware implementations allows comparing their results in order to validate hardware implementation.

IV. METHODOLOGY

A. Function Verification.

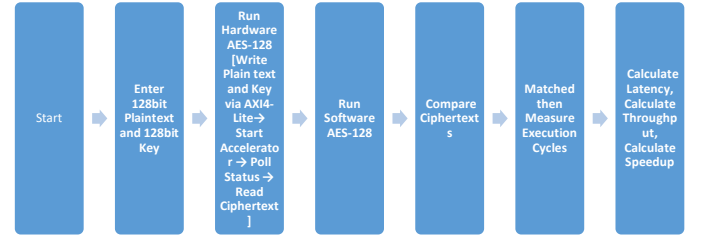
The functionality of the hardware and software was verified by using the same 128bit plain text and 128bit encryption key in both the software and hardware AES-128 encryption algorithms. Software AES-128 encryption algorithm runs on the whole AES-128 algorithm in the MicroBlaze V processor, whereas hardware AES-128 encryption algorithm sends the same plain text data to the AES-128 encryption algorithm through the AXI4-Lite interface. After running both the encryption algorithms, the outputs of 128bit ciphertext were compared. Similar

ciphertext proves the correctness of the hardware AES-128 encryption algorithm.

B. Performance Measurement Procedure

The performance measurements were carried out using the AXI Timer that works at 100 MHz. In the case of the hardware implementation, the timer was activated just before invoking the hardware encryption function and deactivated right after returning from the function call. Consequently, the hardware implementation performance measures the entire transaction of the processor controlled accelerator, which includes sending the input plaintext and encryption key to the AES-128 peripheral, executing the encryption process, waiting for its completion, and receiving the output ciphertext. In the case of the software implementation, the timer was activated right before executing the AES-128 software encryption function and deactivated right after its execution.

Fig. 2. Experimental methodology for AES-128 hardware and software performance evaluation.



C. Performance Metrics

- The measured execution cycle counts were used to calculate latency, throughput, and hardware acceleration speedup. For a measured cycle count N and timer frequency f at **100MHz**, the execution latency was calculated as

$$Latency = \frac{N}{f}$$

- Since AES operates on a 128bit data block, throughput was calculated as,

$$Throughput = \frac{128}{Latency}$$

- The hardware acceleration speedup was calculated from the ratio between the software and hardware execution cycle counts:

$$Speedup = \frac{N(Software)}{N(Hardware)}$$

(N = Number of Cycles)
(f = frequency (100MHz))

V. RESULTS AND DISCUSSION

A. Function Verification

Functionality verification was done using the same 128bit plaintext and encryption key in the hardware AES-128 implementation as well as in the software AES-128 implementation, and the outputs were found to be matched correctly, indicating that the hardware acceleration implementation gave the same encryption output as compared to the software one. Functional verification was done prior to performance evaluation.

Fig. 3. Functional verification of the Hardware and Software implementations.

```

D:\xilinx\2025.2\ >
Terminal requirements :
(1) Processor's STDOUT is redirected to the ARM DCC/MDM UART.
(11) Processor's STDIN is redirected to the ARM DCC/MDM UART.
Then, text input from this console will be sent to DCC/MDM's UART port.
NOTE: This is a line-buffered console and you have to press "Enter"
to send a string of characters to DCC/MDM.

AES-128 HW/SW Comparison

Enter Plaintext (32 hex chars): 12345678912345678912345678912345
2468135792468135
Enter Key (32 hex chars): 98765432198765432198765432198765
65432198765432198765

Hardware Ciphertext = F4C118B24789A8B4DC6FE8F28106566
Software Ciphertext = F4C118B24789A8B4DC6FE8F28106566
Verification = MATCH

Performance Results :
Hardware cycles = 1196
Software cycles = 39888
Hardware latency = 11.960 us
Software latency = 398.880 us
Hardware throughput = 10.702 Mbps
Software throughput = 0.320 Mbps
Speedup = 33.35x
END

Enter Plaintext (32 hex chars):

```

B. Performance Results

The performance analysis of the hardware and software AES-128 implementations is done by using the execution cycle counts measured by the 100 MHz AXI Timer. The hardware implementation took **1,196 cycles**, whereas the software implementation took **39,888 cycles** to encrypt one AES 128-bit block. In terms of latency, it means that the latency of the hardware implementation was **11.960 μ s**, whereas that of the software implementation was **398.880 μ s**. Consequently, the throughput of the hardware accelerator was **10.702 Mbps**, while that of the software implementation was approximately **0.320 Mbps**. With Hardware Execution Speedup of **33.35x**.

TABLE II. AES-128 HARDWARE AND SOFTWARE PERFORMANCE COMPARISON

PERFORMANCE METRIC	HARDWARE	SOFTWARE
EXECUTION CYCLES	1196	39888
LATENCY	11.960	398.880
THROUGHPUT	10.702	0.32
SPEEDUP	33.35x	1x

Fig. 4. Execution-cycle comparison of hardware and software AES-128 implementations.

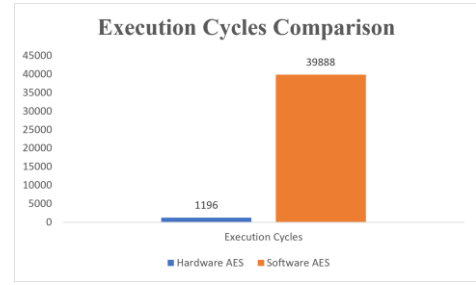


Fig. 5. Execution-latency comparison of hardware and software AES-128 implementations.

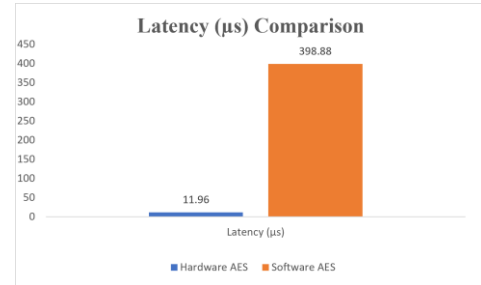
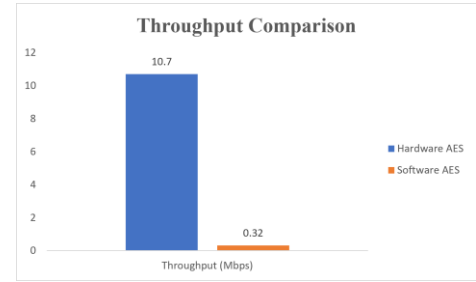


Fig. 6. Throughput comparison of hardware and software AES-128 implementations.



Figures 4 to 6 show the results of performance comparisons of hardware AES-128 implementation and software AES-128 implementation in terms of execution cycles, latency, and throughput. Hardware implementation needs far fewer execution cycles than software implementation, which leads to less latency and more throughput.

C. Discussion

Based on the experimental results, there is evident improvement in the performance of AES-128 encryption algorithm when implemented on the dedicated FPGA hardware accelerator as opposed to pure software running on the MicroBlaze V processor. The reduction of software cycles from 39,888 to 1,196 indicates that the dedicated hardware solution eliminates the instruction-processing overhead involved in software execution. This results in the latency decrease from 398.880 μ s to 11.960 μ s which translates into throughput increase from approximately 0.320 Mbps to 10.702 Mbps.

It can be seen that the speedup is approximately 33.35x. This confirms the advantages of offloading AES-128

encryption algorithm from the processor to the dedicated FPGA logic. However, it should be noted that the measured execution time of the hardware does not comprise only the encryption rounds of AES algorithm. Rather, this time comprises the whole cycle of processor-controlled transaction on the accelerator, including AXI4-Lite data transfer, start, monitoring and fetching of ciphertext. Thus, the speedup is approximately $33.35\times$ at the hardware/software system level rather than only AES encryption core execution speed.

The achieved performance increase is in line with other works in the area of FPGA-based AES accelerator design and implementation where acceleration is achieved by the transfer of complex and computationally expensive cryptographical computations from the processor software to special purpose hardware and thus decreasing the processing time [3] - [8]. However, direct numerical comparison with previously reported acceleration results should be made carefully because the reviewed implementations differ in FPGA devices, processor architectures, operating frequencies, hardware/software partitioning, accelerator interfaces, and measurement methods. The results obtained in this work demonstrate the performance advantage of the proposed AES-128 accelerator relative to the software AES-128 implementation executed on the same MicroBlaze V based FPGA system under the same experimental conditions.

VI. CONCLUSION

In this work, the design, implementation, and performance analysis of AES-128 hardware accelerator were conducted together with MicroBlaze V processor in an FPGA system. AES-128 hardware accelerator was designed using Verilog and interfaced with the MicroBlaze V processor via a customized AXI4-Lite memory mapped peripheral. AES-128 software implementation was also conducted using the same MicroBlaze V processor for consistency in performance. Functional verification of the two AES-128 algorithms involved the use of same plaintext and keys, which resulted in the same ciphertext.

The performance measurements performed via 100 MHz AXI Timer resulted in the value of **1,196** clock cycles for hardware implementation and **39,888** clock cycles for software implementation. Thus, the latencies for hardware and software implementations were measured to be equal to **11.960 μ s** and **398.880 μ s**, respectively, whereas the throughputs amounted to **10.702 Mbps** and **0.320 Mbps**, respectively. In addition, the hardware acceleration provided the speedup ratio equal to **33.35 $\times$** . Besides, the hardware performance measurement involves the calculation of the time for processor-controlled accelerator transaction rather than the internal AES round only, which gives an opportunity to estimate the performance of the whole system with implemented accelerator. This experiment proves that implementing the AES-128 encryption hardware acceleration brings a considerable performance gain comparing to its sequential processor execution while maintaining MicroBlaze V software control.

ACKNOWLEDGEMENT

The author thanks to Dr. Xiaojun Wang for supervision and technical support for this project.

REFERENCES

- [1] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, Federal Information Processing Standards Publication FIPS 197. Doi: <https://doi.org/10.6028/NIST.FIPS.197-upd1>
- [2] AMD, *MicroBlaze V Processor Embedded Design User Guide (UG1711)*, ver. 2024.1, Aug. 2024. <https://docs.amd.com/r/2024.1-English/ug1711-microblaze-v-embedded-design>
- [3] Gomes, T.; Sousa, P.; Silva, M.; Ekpanyapong, M.; Pinto, S. FAC-V: An FPGA-Based AES Coprocessor for RISC-V. *J. Low Power Electron. Appl.* 2022, 12, 50. Doi: <https://doi.org/10.3390/jlpea12040050>
- [4] Azzouzi, O.; Anane, M.; Ghanem, M.C.; Himeur, Y.; Wojtczak, D. Flexible and Area-Efficient Codesign Implementation of AES on FPGA. *Cryptography* 2025, 9, 78. <https://doi.org/10.3390/cryptography9040078>
- [5] Renuka, G., Shree, V., & Reddy, P. (2018). Comparison of AES and DES Algorithms Implemented on Virtex-6 FPGA and Microblaze Soft Core Processor. *International Journal of Electrical and Computer Engineering (IJECE)*, 8(5), 3544-3549. doi:<http://doi.org/10.11591/ijece.v8i5.pp3544-3549>
- [6] A. Zgheib, O. Potin, J. -B. Rigaud and J. -M. Dutertre, "Extending a RISC-V core with an AES hardware accelerator to meet IOT constraints," SMACD / PRIME 2021; International Conference on SMACD and 16th Conference on PRIME, online, 2021, pp. 1-4.
- [7] P. Dixit, J. Zalke and S. Admane, "Speed optimization of aes algorithm with hardware-software co-design," 2017 2nd International Conference for Convergence in Technology (I2CT), Mumbai, India, 2017, pp. 793-798, doi: 10.1109/I2CT.2017.8226237.
- [8] Vilas V Deotare, Dinesh V Padole Ashok S. Wakode., "Performance Evaluation of AES using Hardware and Software Codesign," *IJRITCC*, vol. 2, no. 6, pp. 1638-1643, Jun, 2014. Doi:<https://doi.org/10.17762/ijritcc.v2i6.3226>
- [9] B. S. S. Tejesh and M. Ramakrishnan, "Efficient Hardware Accelerator Design for AES Encryption Using High-Level Synthesis Techniques," *2024 International Conference on Integrated Intelligence and Communication Systems (ICIICS)*, Kalaburagi, India, 2024, pp. 1-6. doi: 10.1109/ICIICS63763.2024.10859678.
- [10] Q. -D. Tran, K. -M. Ma, T. -T. Bui, C. -K. Pham and D. -H. Le, "Implementing a Secure Boot System on a 32-Bit RISC-V CPU on FPGA," *2024 RIVF International Conference on Computing and Communication Technologies (RIVF)*, Danang, Vietnam, 2024, pp. 141-145, doi: 10.1109/RIVF64335.2024.11009032

APPENDIX A USE OF GEN AI

There have been no tools from Generative Artificial Intelligence (GenAI) used in preparing this project portfolio.

Venkatesh Maddukuri
A00016235

APPENDIX B

EEN1101 STATUS REPORT

Page. (B.1 – B.7)

NAME : VENKATESH MADDUKURI
STUDENT ID: A00016235
COURSE : EEN1101 Research Training & Project Planning
UNIVERSITY: DUBLIN CITY UNIVERSITY

THEME : FPGA BASED RISC-V SYSTEM DESIGN WITH HARDWARE
ACCELERATION

SUPERVISOR : DR. PROF. XIAOJUN WANG

**PROJECT TITLE: Design and Performance Evaluation of an AES-192
Hardware Accelerator in a MicroBlaze V Based FPGA
System**

INDEX

PAGE	CONTENT
2	DESCRIPTION
2-3	BACKGROUND INFORMATION
3	OBJECTIVE
4	CORE
4	DESIGN WORK
5	PROJECT PLANNING
6	REFERENCES

Description:

In today's digital world, a big amount of sensitive information like financial information, personal information, and secure communication are being transmitted through networks. The major challenge here is to protect the information from unauthorized access. One of the most commonly used encryption techniques to protect digital information is the Advanced Encryption Standard, which offers high security to information and is used in many communication and embedded systems.

The AES encryption technique offers three key sizes: 128, 192 and 256 bits. However, in case of software implementation of AES encryption techniques in a processor, especially in case of a soft-core processor in FPGA technology, the encryption process may take time due to its complexity. To overcome the above limitation, hardware acceleration can be used. If the AES operations are performed as a hardware accelerator on an FPGA, the encryption operation can be performed in parallel, thus improving the performance of the system.

Therefore, the primary problem that is being addressed in the project is the limitation of the performance of the software-based execution of the AES operation. The project aims to design a system that can perform the comparison of the software-based execution of the AES operation on the processor and the hardware-based execution of the AES operation on the FPGA.

Background information:

The block size of the AES algorithm is fixed, and it supports three different key sizes those are 128, 192 and 256-bit keys. Each of the different key sizes has a different number of encryption rounds. The number of encryption rounds in the AES algorithm is 10 for 128-bit keys, 12 rounds for 192-bit keys, and 14 rounds for 256-bit keys.

The encryption algorithm can be performed by running the software on the processor with any computer language. However, the performance of the software may be limited by the sequence of operations performed in the encryption algorithm.

Another approach is to use hardware acceleration, where a cryptographic algorithm is implemented on a FPGA, where a number of operations can be performed in parallel, that improves performance when compared to software implementation. However, most of the previous works focus majorly on AES-128 key size implementations or highly optimized hardware architectures without comparing the performance between software execution and hardware acceleration in embedded FPGA systems.

Several research studies have been conducted on the implementation of AES on FPGAs and hardware accelerators in cryptographic systems.

Some research on the real-time FPGA implementation of AES has shown that hardware-based encryption systems improve performance in comparison to software implementations. The research has shown that FPGA architectures are capable of implementing AES operations such as SubBytes, ShiftRows, MixColumns, and AddRoundKey.

Other research studies on AES implementation are focused on optimizing AES implementations to minimize hardware resources while improving performance. Some research has been conducted on pipelined architectures and S-box implementations to improve the performance of AES encryption systems.

Although a lot of research works have been carried out to study the implementation of AES in FPGA technology, it is still necessary to study the comparison between software implementation and hardware acceleration in the FPGA system. It has been observed that AES-

192 implementations have been less discussed in comparison to AES-128 implementations, although AES-192 provides a high level of security with moderate complexity in designing.

OBJECTIVES:

1. To understand the working principles of AES encryption algorithm.
2. To develop AES-192 encryption algorithm in software for a processor MicroBlaze V (it is nothing but softcore processor of RISC - V that cover my theme).
3. To develop a hardware accelerator for AES-192 encryption algorithm in FPGA using Verilog.
4. To integrate the hardware accelerator with the microBlaze V processor.
5. To compare software implementation with hardware acceleration in terms of performance and efficiency.

CORE:

The project demonstrates how hardware acceleration improves cryptographic performance. Two implementations will be compared:

Software Implementation

AES-192 executed on the MicroBlaze processor and note the performance terms.

Hardware Accelerator Implementation

AES-192 implemented as custom FPGA hardware logic using vivado and connected to the processor by using AXI.

Then we compare these performance terms:

- Execution time
- Throughput

How The Design Works:

This proposed system is implemented on the FPGA platform, and a processor accelerator architecture is used in the implementation. The MicroBlaze soft core processor is created in the FPGA design environment, and it is used as the main controller in the system. The AES-192 encryption algorithm is first implemented in software and executed on the processor to measure the performance in terms of execution time and throughput. After that a hardware accelerator is designed for the AES-192 encryption algorithm in a hardware description language Verilog, and it is connected to the processor using the AXI communication interface. The hardware accelerator is used to improve the performance in terms of execution time and throughput by executing the encryption operation more efficiently in parallel using hardware components. At the same time, the data, key, and encrypted data are stored in on-chip memory, such as Block RAM, in the FPGA system. The system is implemented on a FPGA, and the performance results are achieved from the implementation. The

performance of the execution of the AES 192 encryption algorithm in software is compared with that in hardware to measure the improvement in performance in terms of execution time and throughput.

Project Planning:

The work will be divided into phase wise that includes the following.

1. AES system design
2. Software implementation
3. Hardware accelerator development
4. Integration
5. Testing performance evaluation.

PHASE	TASK	DURATION
System Design	Create architecture diagram	Week 1
	Plan memory usage & data flow	Week 2
Software implementation	Develop AES-192 code in C	Week 3 and 4
	Run software on processor	Week 4
	Measure baseline performance	Week 4
Hardware accelerator development	Develop AES modules (HDL)	Week 5, 6
	Implement core AES functions	Week 6,7
Integration	Integrate accelerator via AXI	Week 8
	Test and Measure performance	Week 9
Testing and performance evaluation.	Test encryption both implementations	Week 10
	Measure execution, throughput, resource	Week 10
	Compare software vs hardware	Week 11
Final Report and Document		Week 12

REFERENCES

1. Ludovico Poli, Sangeet Saha, Xiaojun Zhai, Klaus D. McDonald-Maier., “Design and implementation of a RISC V processor on FPGA,” *IEEE Conference Publication | IEEE Xplore*, Dec. 01, 2021. DOI: 10.1109/MSN53354.2021.00037.
2. Pravin S. Mane, Indra Gupta, M. K. Vasantha., “Implementation of RISC processor on FPGA,” *IEEE Conference Publication | IEEE Xplore*, Dec. 01, 2006. DOI: 10.1109/ICIT.2006.372448
3. Quoc-Dang Tran, Khai-Minh Ma, Trong-Tu Bui, Cong-Kha Pham and Duc-Hung Le., “Implementing a secure boot system on a 32-Bit RISC-V CPU on FPGA,” *IEEE Conference Publication | IEEE Xplore*, Dec. 21, 2024. **DOI:** 10.1109/RIVF64335.2024.11009032
4. Khai-Duy Nguyen, Dang Tuan Kiet, Trong-Thuc Hoang, Nguyen Quang Nhu Quynh, Xuan-Tu Tran, and Cong-Kha Pham., “A trigonometric hardware acceleration in 32-bit RISC-V microcontroller with custom instruction,” *IEICE Electronics Express*, vol. 18, no. 16, p.20210266, Jul. 2021. DOI: 10.1587/elex.18.20210266.
5. Pushkar R Kulkarni, Manjunath, Priyansh Rajiv Dhotar, Umang Mishra and Mekhala Vinod Purohit., “FPGA-Based Hardware Accelerator with RISC-V Processor for Real-Time CNN Inference on ECG Signals,” *IEEE Conference Publication | IEEE Xplore*, Nov. 20, 2025. DOI: 10.1109/CSITSS67709.2025.11294741
6. Mazen EI Maraghi, Salma Hesham and Mohamed A. Abd EI Ghani., “Real-time efficient FPGA implementation of aes algorithm,” *IEEE Conference Publication | IEEE Xplore*, Sep. 01, 2013. DOI: 10.1109/SOCC.2013.6749688
7. M. Wygrzvwalski, P. Skrzypiec, and R. Szczygiel, “Hardware acceleration method using RISC-V core with no ISA extensions,” *IEEE XPLORE*, Jun. 27, 2024, pp. 265–269. DOI: 10.23919/mixdes62605.2024.10614026.
8. T. Gomes, P. Sousa, M. Silva, M. Ekpanyapong, and S. Pinto, “FAC-V: an FPGA-Based AES coprocessor for RISC-V,” *Journal of Low Power Electronics and Applications*, vol. 12, no. 4, p. 50, Sep. 2022, DOI: 10.3390/jlpea12040050.
9. A. Gielata, P. Russek, and K. Wiatr, “AES Hardware implementation in FPGA for Algorithm Acceleration Purpose,” *IEEE Xplore*, pp. 137–140, Jan. 2008, DOI: 10.1109/icses.2008.4673377.
10. F. Restuccia, A. Biondi, M. Marinoni, G. Cicero, and G. Buttazzo, “AXI HyperConnect: a predictable, hypervisor-level interconnect for hardware accelerators in FPGA SOC,” *IEEE Xplore*, Jul. 2020, doi: 10.1109/dac18072.2020.9218652.

APPENDIX C

PROJECT PLAN, PROGRESS, AND ACHIEVEMENTS

C.1 Original Approved Plan

The initial project plan created in EEN1101 Research and Development module suggested carrying out the research and implementation of hardware acceleration of cryptography AES-192 on an embedded system based on FPGA technology. After that changed to AES-128 before EEN105 module starts after consultations with the project supervisor. This plan set forward the initial goals of the project, the methodology of its implementation and the way it should be tested.

C.2 Progress and Outcomes

This project went through various stages of research, hardware development, and integration. First, the AES-128 cipher algorithm and existing hardware accelerators based on FPGAs were researched in order to build the proper system architecture. The AES-128 Encryption Accelerator was implemented using Verilog and designed to encrypt a 128-bit plaintext block using 128-bit encryption key. After testing the functionality of the Encryption Accelerator, it was included in the customized AXI4-Lite peripheral in order to work with the MicroBlaze V processor. The 128-bit plaintext and encryption key were transferred via four 32-bit memory mapped registers each, while control and status registers were used for encryption start and completion detection respectively.

Software Development involved control of the hardware accelerator using the MicroBlaze V processor and creation of software AES-128 cipher for comparison purposes. An AXI Timer running at 100MHz was added to help determine the cycle counts of both implementations under the same input parameters. Functionality testing showed that both hardware and software AES-128 implementations yielded identical ciphertexts given identical plaintexts and encryption keys. Performance testing showed that hardware-accelerated implementation performed 1,196 cycles of processing while software AES-128 cipher required 39,888 cycles. Latencies were thus found to be 11.960 μ s and 398.880 μ s, respectively, showing that the hardware-accelerated implementation is about 33.35 times faster than software implementation.

Project Stage	Outcome
Background research	AES-128, FPGA based acceleration, HW/SW Co-Design and processor-based cryptography studied
Software development	Development of processor controller and software AES-128 implementation
AES hardware development	Verilog implementation of AES-128 encryption accelerator
Functional testing	Verification of AES hardware functionality
AXI4-Lite integration	Custom designed memory mapped AES-128 peripheral
Timing implementation	Added 100 MHz AXI Timer to IP Block design to measure execution cycles
HW/SW verification	Hardware & software ciphertexts matched successfully
Performance evaluation	1,196 HW cycles against 39,888 SW cycles

Final outcome	Around 33.35x hardware acceleration achieved
---------------	--

C.3 Major Changes to Scope or Methodology

The first major change in the initial project proposal is in relation to AES-192 and AES-128 encryption methods that have been used for the implementation. AES-192 had been initially proposed as the encryption method to be used in both software and hardware implementation on MicroBlaze V and Verilog respectively. This change was carried out before this EEN1095 module starts with implementation, after consultations with the project supervisor. The use of AES-192 was viewed to be complex in design due to its large key size and the larger number of encryption rounds. It was then decided to use AES-128 according to recommendations by the project supervisor. This will not hinder the main aim of the project that is to investigate the performance gain that can be gained from using FPGAs in AES hardware acceleration.

The system architecture was also changed slightly upon based on the first set of proposals with status report. The original proposal had a connection between the hardware accelerator and the processor through AXI and also the application of on-chip memory like Block RAM to store the input data, encryption key, and ciphertext. In the final design, the MicroBlaze V processor to AES-128 accelerator communication was done through an AXI4-Lite memory-mapped peripheral where 128-bit plain text, key, and ciphertext were sent via groups of 32-bit registers together with control and status registers. An AXI Timer running at 100 MHz was included for performance measurement purposes. All this was achieved while maintaining the original idea of comparing performance between software and hardware.

C.4 Achievements Against the Original Plan from EEN1101 Project Plan

Objectives(EEN1101)	Achievement
Understand the working principles of the AES encryption algorithm.	Achieved. The AES encryption algorithm was analysed, covering the processes of SubBytes, ShiftRows, MixColumns, AddRoundKey, and key expansion necessary for its hardware and software implementation.
Develop AES-192 encryption in software for the MicroBlaze V processor.	Achieved with a change. Instead of AES-192, AES-128 was chosen in the research phase after consulting with the supervisor. AES-128 software implementation was created in C and run on the MicroBlaze V processor.
Develop an AES-192 hardware accelerator in FPGA using Verilog.	Achieved. AES-128 encryption accelerator design was done in Verilog language and implemented in the FPGA architecture.
Integrate the hardware accelerator with the MicroBlaze V processor.	Achieved. The AES-128 accelerator was integrated with the MicroBlaze V processor as a custom AXI4-Lite memory-mapped peripheral using plaintext, key, ciphertext, control, and status registers.
	Achieved. Both the implementations were tested by providing same inputs and evaluated with a 100 MHz AXI timer.

Compare software implementation with hardware acceleration in terms of performance and efficiency.

Hardware took 1,196 clock cycles, while software implementation took 39,888 clock cycles, which gave a speed-up of about 33.35 times.

C.5 — Key References for Detailed Assessment

Reference Number in Paper	Full Citation	Relevance
[1]	National Institute of Standards and Technology, <i>Advanced Encryption Standard (AES)</i> , Federal Information Processing Standards Publication FIPS 197. Doi: https://doi.org/10.6028/NIST.FIPS.197-upd1	It is the main technical reference for the AES algorithm employed by the project. It describes the 128-bit block structure of AES, the supported key sizes, number of rounds, SubBytes, ShiftRows, MixColumns and AddRoundKey transforms, and the key expansion process. It was therefore the standard for implementing the AES-128 algorithm in both hardware and software in the course of the project. Unlike the research papers, it does not suggest any architecture of the FPGA but serves as the standard for AES function implemented in the developed accelerator.
[3]	Gomes, T.; Sousa, P.; Silva, M.; Ekpanyapong, M.; Pinto, S. FAC-V: An FPGA-Based AES Coprocessor for RISC-V. <i>J. Low Power Electron. Appl.</i> 2022, 12, 50. Doi: https://doi.org/10.3390/jlpea12040050	It is among the closest references because it analyses an FPGA AES accelerator co-processor that is implemented in conjunction with a RISC-V processor for AES-128, AES-192, and AES-256. In particular, it compares the tightly-coupled and loosely-coupled accelerator design approaches. Loose coupling relies on a memory-mapped technique for the communication between the processor and accelerator, which can be viewed as close to the AXI4-Lite memory-mapped architecture employed in this project. The major difference lies in the fact that this project utilizes MicroBlaze V along with a custom AXI4-Lite peripheral.
[4]	Azzouzi, O.; Anane, M.; Ghanem, M.C.; Himeur, Y.; Wojtczak, D. Flexible and Area-Efficient Codesign Implementation of AES on FPGA. <i>Cryptography</i> 2025, 9, 78. https://doi.org/10.3390/cryptography9040078	The relevance of this reference lies in the fact that it deals with hardware/software co-design of AES using a MicroBlaze based FPGA platform. It gives insight into the implementation of AES processing by partitioning it among the processor software and hardware FPGA. This gives a good contrast with the architecture used in this project. The only point of difference between the two architectures lies in the hardware/software partition where all AES-128 encryption processing is done through the Verilog accelerator in this project whereas MicroBlaze V manages other aspects of the system.

[5]	Renuka, G., Shree, V., & Reddy, P. (2018). Comparison of AES and DES Algorithms Implemented on Virtex-6 FPGA and Microblaze Soft Core Processor. International Journal of Electrical and Computer Engineering (IJECE), 8(5), 3544-3549. doi:http://doi.org/10.11591/ijece.v8i5.pp3544-3549	The current work is highly relevant to the primary objective of the performance evaluation of the project since it involves cryptographic implementation through dedicated FPGA hardware and the use of a soft-core processor MicroBlaze. It explains the reason why the study is motivated to determine if dedicated hardware performs better than software implementations. The current project uses a similar general approach of comparing hardware and software implementations but applies to AES-128 with a MicroBlaze V processor and an AXI4-Lite accelerator. A direct comparison of numerical results cannot be performed due to the differences between the two projects.
[7]	P. Dixit, J. Zalke and S. Admane, "Speed optimization of aes algorithm with hardware-software co-design," 2017 2nd International Conference for Convergence in Technology (I2CT), Mumbai, India, 2017, pp. 793-798, doi: 10.1109/I2CT.2017.8226237.	The above-referenced paper is highly relevant to the topic of AES improvement by means of HW/SW codesign. It deals with software-based AES execution and computation offloading onto the FPGA hardware in order to increase speed of execution. The above-mentioned reference serves as a background for the current project idea in which AES encryption is offloaded from the MicroBlaze V processor to FPGA hardware. However, the current research is different in the sense that a full AES-128 Verilog accelerator is used as a custom AXI4-Lite peripheral in contrast to the software-only AES-128 in MicroBlaze V processor.

APPENDIX D

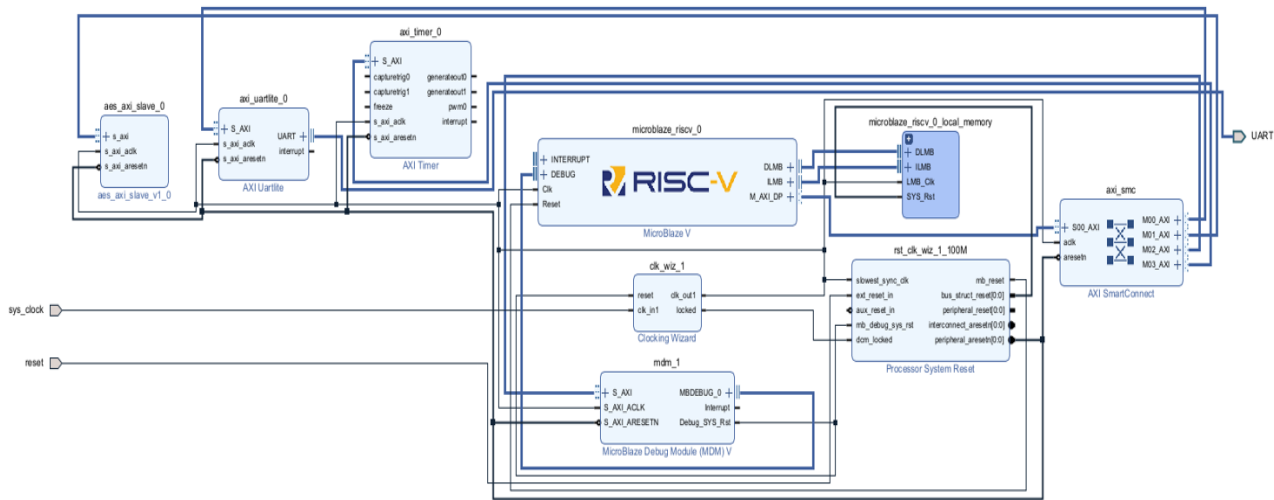
DESIGN AND IMPLEMENTATION DETAILS

D.1 Overall FPGA System Architecture and Hardware Integration

The proposed system comprises a MicroBlaze V soft core, a custom AES-128 hardware accelerator, AXI4-Lite interface, AXI Timer, and UART communication on the FPGA based platform. The MicroBlaze V acts as the system controller and is used for handling all the functions including taking the input message as plaintext and the encryption key, handling the hardware accelerator, performing software AES-128 algorithm, and showing the results of the encryption and performance. The AES-128 hardware accelerator is designed to perform the AES-128 encryption completely using FPGA logic, whereas communication between processor and accelerator is done using the custom AXI4-Lite interface.

The process of hardware encryption includes writing of the 128-bit message and the 128-bit encryption key into the AES peripheral through 32-bit registers and writing to the control register for encryption initiation by the MicroBlaze V processor. The AES-128 accelerator does the encryption and notifies of its end through the status register after that; the 128-bit encrypted message is read from four 32-bit output registers. The AES peripheral is mapped to the address space of the processor with the base address equal to 0x80000000. For evaluation of the performance metrics of the alternative implementation path, AES-128 is implemented in software on the MicroBlaze V processor with the use of the same input parameters. The AXI Timer with a clock frequency of 100 MHz is used to get the execution cycle measurements.

Fig. D.1. Xilinx Vivado block design of the implemented MicroBlaze V based AES-128 hardware acceleration system.



D.2 AES-128 Hardware Accelerator and AXI4-Lite Interface

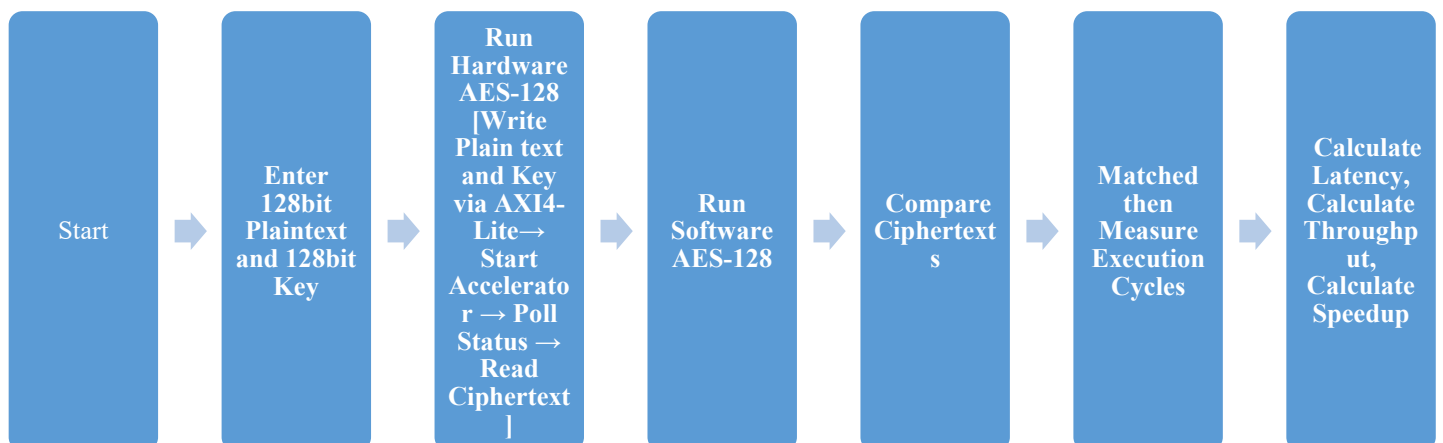
The AES-128 Hardware Accelerator was designed in Verilog for the purpose of executing the entire encryption of a 128-bit plaintext block, which is done using a 128-bit encryption key. The process of encryption starts with the AddRoundKey function first, followed by a series of ten AES encryption rounds. Rounds 1 to 9 consist of SubBytes, ShiftRows, MixColumns, and AddRoundKey functions, whereas in the tenth round, there is no MixColumns function executed. All the necessary round keys are extracted from the original 128-bit encryption key, as part of the AES-128 key expansion algorithm.

In order to enable control of the accelerator by the MicroBlaze V processor, the AES-128 core was added to a custom AXI4-Lite memory mapped peripheral. Since the data bus used to connect the peripheral by the AXI4-Lite is 32 bits in width, both the 128-bit plaintext and encryption key were split into four 32-bit chunks and stored into specific peripheral registers. The processor controls the start of the encryption by use of a control register, while a status register enables the processor to check whether the accelerator has finished processing the input data. Once the encryption process is finished, the resulting 128-bit ciphertext is available using four 32-bit output registers accessible to the processor.

AES-128 accelerator register-map table

Register Group	Width	Function
Plaintext Registers	128 bits	Stores plaintext input
Key Registers	128 bits	Stores AES-128 encryption key
Ciphertext Registers	128 bits	Stores encrypted output
Control Register	32 bits	Starts encryption
Status Register	32 bits	Indicates encryption completion

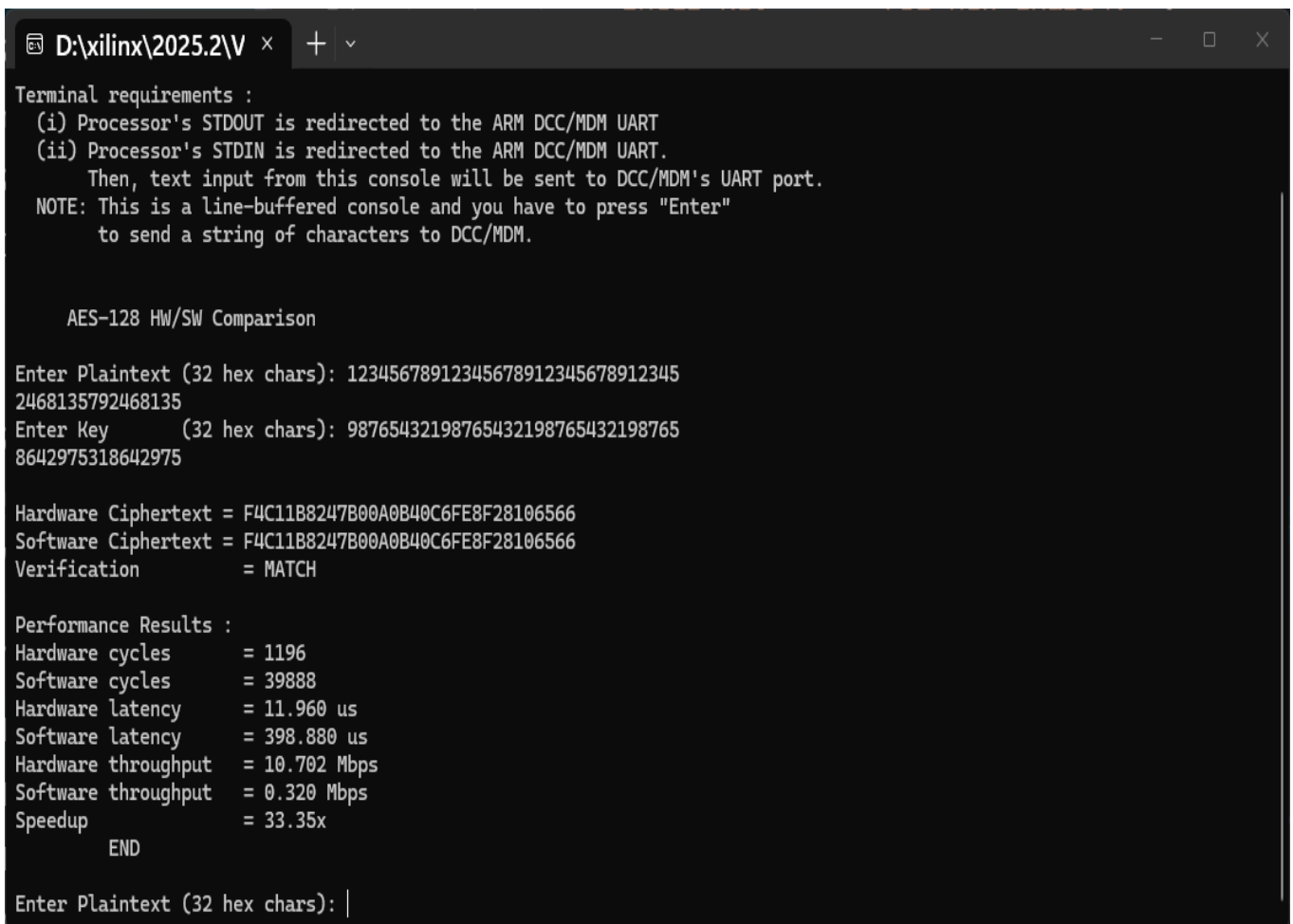
AES encryption flow diagram



D.3 Software Control, Verification, and Performance Measurement

The MicroBlaze V program is written in C language in order to control the AES-128 hardware accelerator as well as to perform the software implementation of the algorithm, which is taken into account for the comparison. Plaintext and encryption key values are obtained through the UART interface and saved in the form of 128-bit data blocks. As far as hardware encryption is concerned, the software sends the plaintext and encryption key to the appropriate AXI4-Lite registers, starts the accelerator using the control register, checks the completion state, and obtains the ciphertext value from the four 32-bit ciphertext registers. The software AES-128 implementation works with the plaintext and encryption key on the MicroBlaze V processor.

Performance measurement was carried out using the AXI Timer running at 100 MHz. In the case of hardware implementation, the timing includes the entire processor-controlled hardware path, from writing the AXI register values, running the accelerator, status checking, and reading of the ciphertext. In the case of software AES-128 execution, the time taken is also measured using the same timer. The results obtained from counting the number of cycles are then used in calculating the latency and throughput, whereas the ratio of the number of cycles between software and hardware is used for the speedup calculations. The application also outputs the hardware and software ciphertexts, verification result, number of execution cycles, latency, throughput, and speedup via UART communication.



```
D:\xilinx\2025.2\V x + v
Terminal requirements :
(i) Processor's STDOUT is redirected to the ARM DCC/MDM UART
(ii) Processor's STDIN is redirected to the ARM DCC/MDM UART.
    Then, text input from this console will be sent to DCC/MDM's UART port.
NOTE: This is a line-buffered console and you have to press "Enter"
      to send a string of characters to DCC/MDM.

AES-128 HW/SW Comparison

Enter Plaintext (32 hex chars): 12345678912345678912345678912345
2468135792468135
Enter Key      (32 hex chars): 98765432198765432198765432198765
8642975318642975

Hardware Ciphertext = F4C11B8247B00A0B40C6FE8F28106566
Software Ciphertext = F4C11B8247B00A0B40C6FE8F28106566
Verification       = MATCH

Performance Results :
Hardware cycles     = 1196
Software cycles     = 39888
Hardware latency    = 11.960 us
Software latency    = 398.880 us
Hardware throughput = 10.702 Mbps
Software throughput = 0.320 Mbps
Speedup             = 33.35x
END

Enter Plaintext (32 hex chars): |
```

Fig. D.3. UART output showing AES-128 hardware/software verification and performance measurements.

APPENDIX E

TESTING AND SUPPLEMENTARY RESULTS

E.1 AES-128 Hardware Accelerator Functional Verification

Functionality verification was done on the AES-128 encryption core through behavioral simulation in Vivado prior to system evaluation of the processor-based implementation. Known 128-bit plaintext and encryption key values were supplied to the encryption core written in Verilog and the output ciphertext was compared against the expected ciphertext value for AES-128 encryption. Simulation was done using the plaintext value 00112233445566778899aabbccddeeff with encryption key 000102030405060708090a0b0c0d0e0f. The hardware implementation produced a ciphertext value 69c4e0d86a7b0430d8cdb78070b4c55a that matched the expected ciphertext.

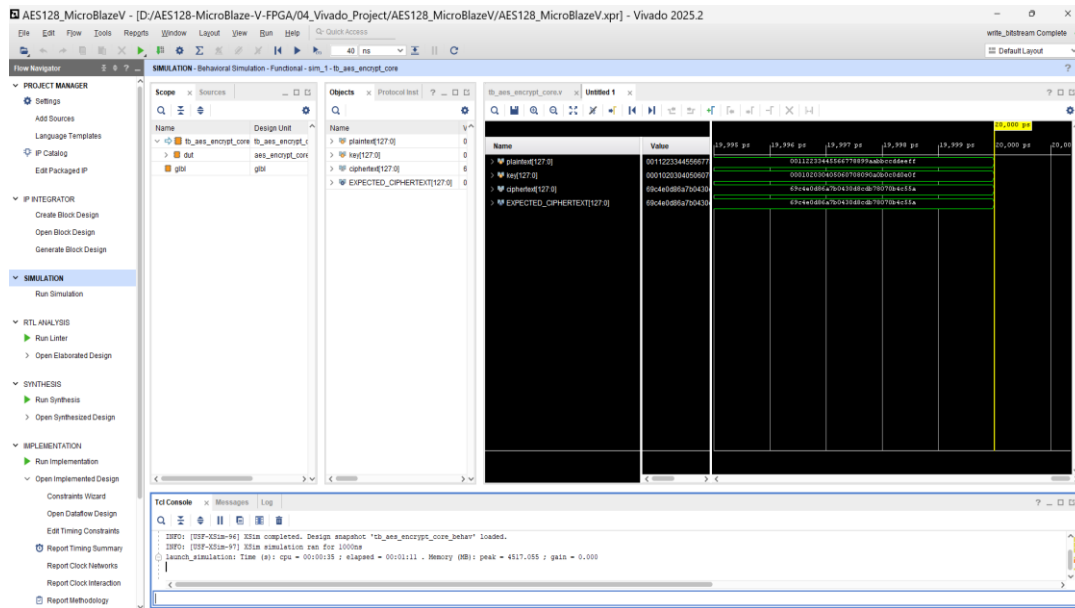


Fig. E.1. Vivado behavioral simulation showing functional verification of the AES-128 encryption core.

The designed FPGA solution was analyzed also through the usage of the Vivado hierarchical resource utilization report. The overall MicroBlaze V-based system required 12,721 Slice LUTs and 2,962 Slice Registers. As for the custom AES AXI slave hierarchy, it required 10,002 Slice LUTs and 465 Slice Registers. Dedicated Block RAM resources were not used in the custom AES AXI hierarchy. The obtained information provides the evidence of the specific FPGA resources required for the custom AES accelerator peripheral, distinguishing them from the FPGA resources required by other parts of the system.

Table E.1. Hierarchical resource utilization of the implemented FPGA system.

Design Component	Slice LUTs	Slice Registers	BRAM
Complete FPGA system	12721	2962	16
AES AXI slave	10002	465	0
MicroBlaze V	1634	1327	0
AXI SmartConnect	448	492	0
AXI Timer	291	240	0
AXI UART Lite	93	111	0

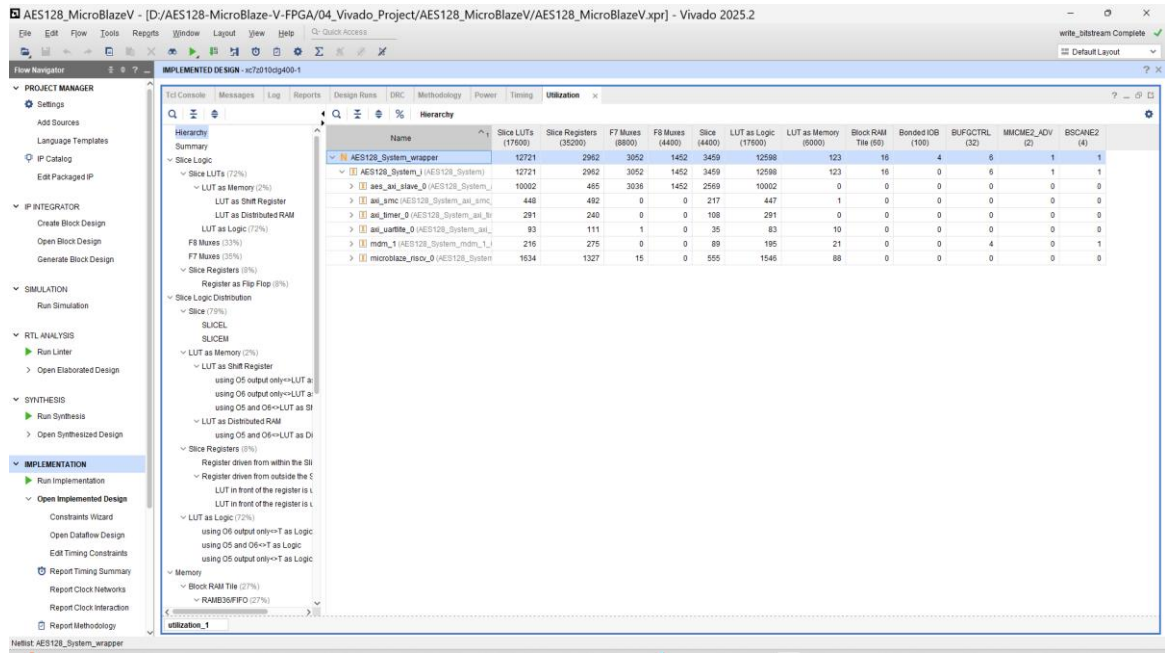


Fig. E.2. Hierarchical FPGA resource utilization of the implemented MicroBlaze V and AES-128 accelerator system.

E.2 Complete System Verification Using Vitis

After programming of the FPGA, testing of the AES-128 hardware/software system was done using the V application implemented for MicroBlaze V in Vitis. This application is fed with 128 bits of plaintext and a 128bit encryption key via the UART interface. In the case of hardware encryption, these two pieces of data are transferred from MicroBlaze V processor to the custom AES-128 peripheral via the AXI4-Lite interface and the encryption process is completed. Finally, the same plaintext and key are encrypted using the software AES-128 on the MicroBlaze V processor.

The ciphertext produced by the hardware accelerator was compared with the ciphertext generated by the software implementation using the application-level verification routine. The UART output displayed both ciphertext values and reported **Verification = MATCH** when the results were identical. This system-level test verified not only the AES-128 encryption functionality but also the complete communication path between the MicroBlaze V processor and the custom AXI4-Lite accelerator, including input transfer, accelerator control, completion monitoring, and ciphertext retrieval.

```

D:\xilinx\2025.2\V x + v
Terminal requirements :
(i) Processor's STDOUT is redirected to the ARM DCC/MDM UART
(ii) Processor's STDIN is redirected to the ARM DCC/MDM UART.
Then, text input from this console will be sent to DCC/MDM's UART port.
NOTE: This is a line-buffered console and you have to press "Enter"
to send a string of characters to DCC/MDM.

AES-128 Hw/Sw Comparison

Enter Plaintext (32 hex chars): 12345678912345678912345678912345
2468135792468135
Enter Key (32 hex chars): 98765432198765432198765432198765
8642975318642975

Hardware Ciphertext = F4C11B8247B0A0B40C6FE8F28106566
Software Ciphertext = F4C11B8247B0A0B40C6FE8F28106566
Verification = MATCH

Performance Results :
Hardware cycles = 1196
Software cycles = 39888
Hardware latency = 11.960 us
Software latency = 398.880 us
Hardware throughput = 10.702 Mbps
Software throughput = 0.320 Mbps
Speedup = 33.35x
END

Enter Plaintext (32 hex chars): |

```

Fig. E.3. UART output shows the AES-128 hardware and software implementations.

E.3 Hardware and Software Performance Results

After the successful performance of functional verification, the performance of the AES-128 hardware accelerator is benchmarked against that of the software-based AES-128 algorithm implementation on the MicroBlaze V processor. Both the implementations are analysed using the same set of plaintext and encryption key. The execution cycles are counted using the AXI Timer, which runs at a frequency of 100 MHz. In the case of the hardware implementation, the time period considered for measurement includes the entire operation controlled by the processor in the hardware, which involves the transfer of plaintext and key via AXI4-Lite, starting of the accelerator, polling of the status, encryption, and the fetching of the ciphertext.

While the hardware implementation needed 1,196 clock cycles, the software implementation took 39,888 clock cycles. Given the timer's frequency of 100 MHz, the latencies would be 11.960 μ s and 398.880 μ s, respectively. The throughput of the hardware implementation is about 10.702 Mbps, while the throughput of the software implementation is 0.320 Mbps. Using the cycle counts reported, the performance gain of the hardware implementation is about 33.35 times faster than software implementation. In conjunction with the ciphertext output shown in **Fig. E.3.**, these results indicate that the designed accelerator performs the required AES-128 operation with reduced latency in system-level encryption operation.

Table E.2. Measured hardware and software AES-128 performance results

Performance Metric	Hardware AES-128	Software AES-128
Execution cycles	1,196	39,888
Latency	11.960 μ s	398.880 μ s
Throughput	10.702 Mbps	0.320 Mbps
Ciphertext verification	MATCH	MATCH
Speedup	33.35×	1x

APPENDIX F

PROJECT REPOSITORY

Loop submission: A00016235_Project_repository.zip

Core items for detailed review:

1. **README.md:** Overview of repository structure, selected core items, and instructions for use.
2. **01_Software_AES128:** MicroBlaze V C application containing hardware accelerator control, software AES-128 implementation, UART interaction, verification, and performance measurement.
3. **02_Hardware_Verilog:** Verilog source files implementing the AES-128 hardware encryption accelerator, including the AES encryption operations and key expansion.
4. **03_AXI_Integration:** Custom AXI4-Lite peripheral source files used to interface the AES-128 accelerator with the MicroBlaze V processor.
5. **04_Vivado_Project:** Complete Vivado FPGA project containing the MicroBlaze V system, custom AES-128 AXI4-Lite peripheral, AXI Timer, and associated FPGA design files.
6. **05_Results:** Hardware and software performance results, including execution cycles, latency, throughput, speedup, and FPGA resource-utilization results.
7. **06_Documentation:** Project documentation including Portfolio.